De un IDOR en el Login a Root: Enumerando, Crackeando y Rematando con un Hash Olvidado en el Código Fuente

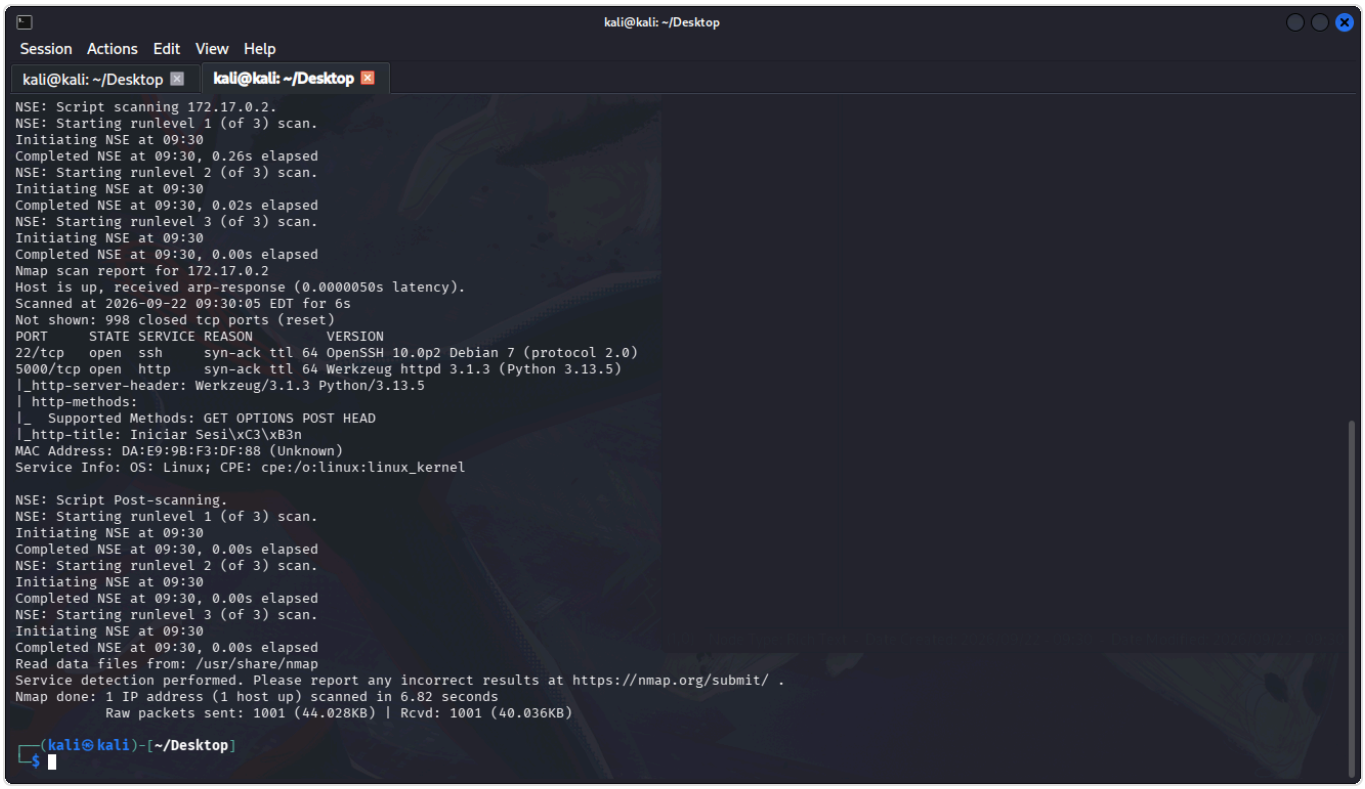
DockerLabs Máquina: Aidor · IP: 172.17.0.2

Writeup de una máquina Linux con una aplicación web en Flask expuesta en el puerto 5000. El fallo de entrada es un **IDOR** de manual en un endpoint de dashboard que filtra los datos — y el hash de la contraseña — de cualquier usuario del sistema sin pedir autenticación. A partir de ahí: automatizar la extracción, crackear en masa, entrar por SSH, y rematar con un segundo hash que quedó olvidado dentro del propio código fuente de la aplicación.

Reconocimiento

Escaneo inicial de puertos y servicios:

```
nmap -sV -sC 172.17.0.2
```



Dos puertos abiertos:

- **22/tcp** — OpenSSH 10.0p2 (Debian)

- **5000/tcp** — Werkzeug/Python (Flask), con el título `Iniciar Sesión`

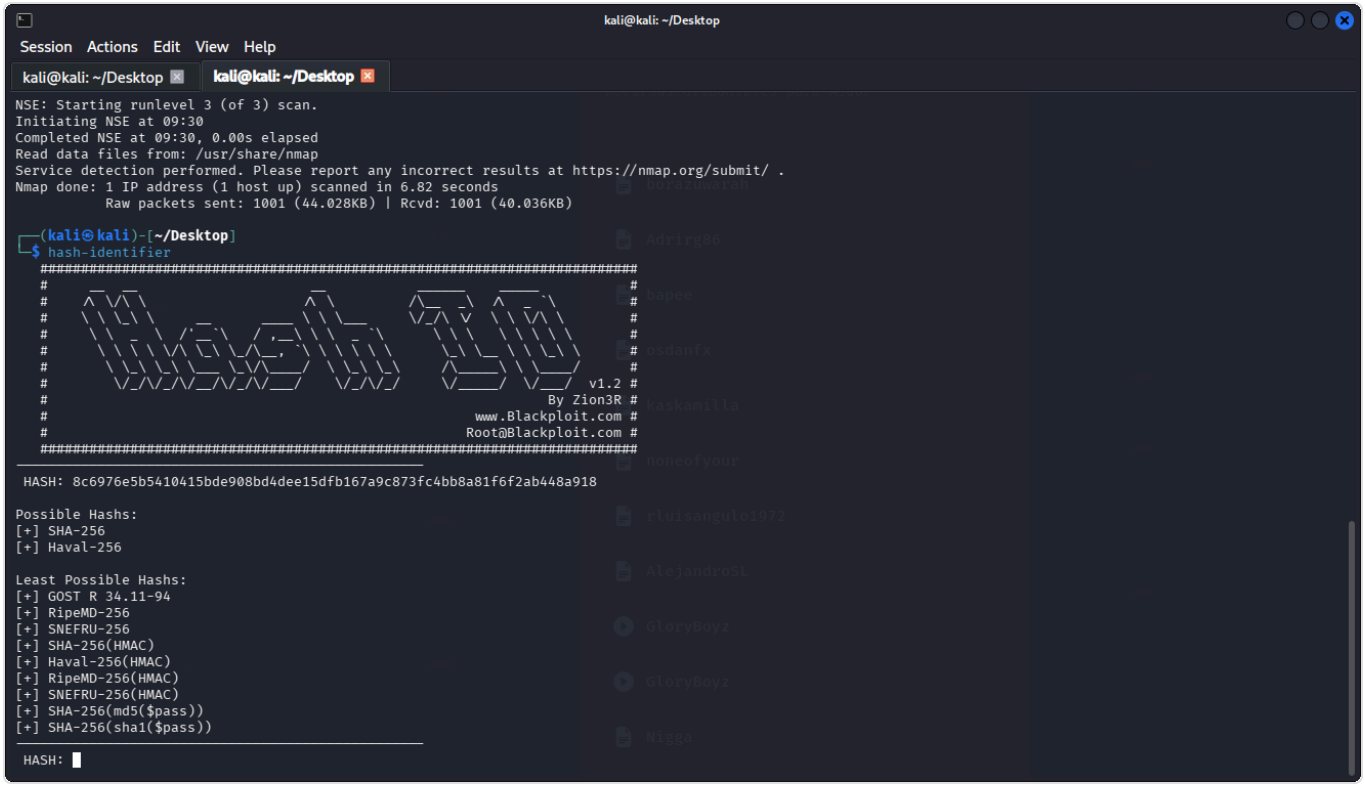
El 5000 con Werkzeug es casi siempre sinónimo de una app Flask en desarrollo, y el título de la página confirma un formulario de login clásico: usuario + contraseña contra una base de datos.

El IDOR: un dashboard que no comprueba nada

La aplicación exponía un endpoint `/dashboard?id=<n>` que devolvía la ficha de un usuario — nombre, email, estado de la cuenta y **el hash de su contraseña** — sin verificar en ningún momento que quien pedía el `id` fuera el dueño de esa cuenta, ni siquiera que estuviera autenticado. Es la definición exacta de un **IDOR (Insecure Direct Object Reference)**: el identificador es secuencial y predecible, y el servidor confía en él a ciegas.

Antes de automatizar nada, confirmé el algoritmo de hash: parecía SHA-256 por longitud, y lo verifiqué hasheando localmente una contraseña conocida y comparando el resultado.

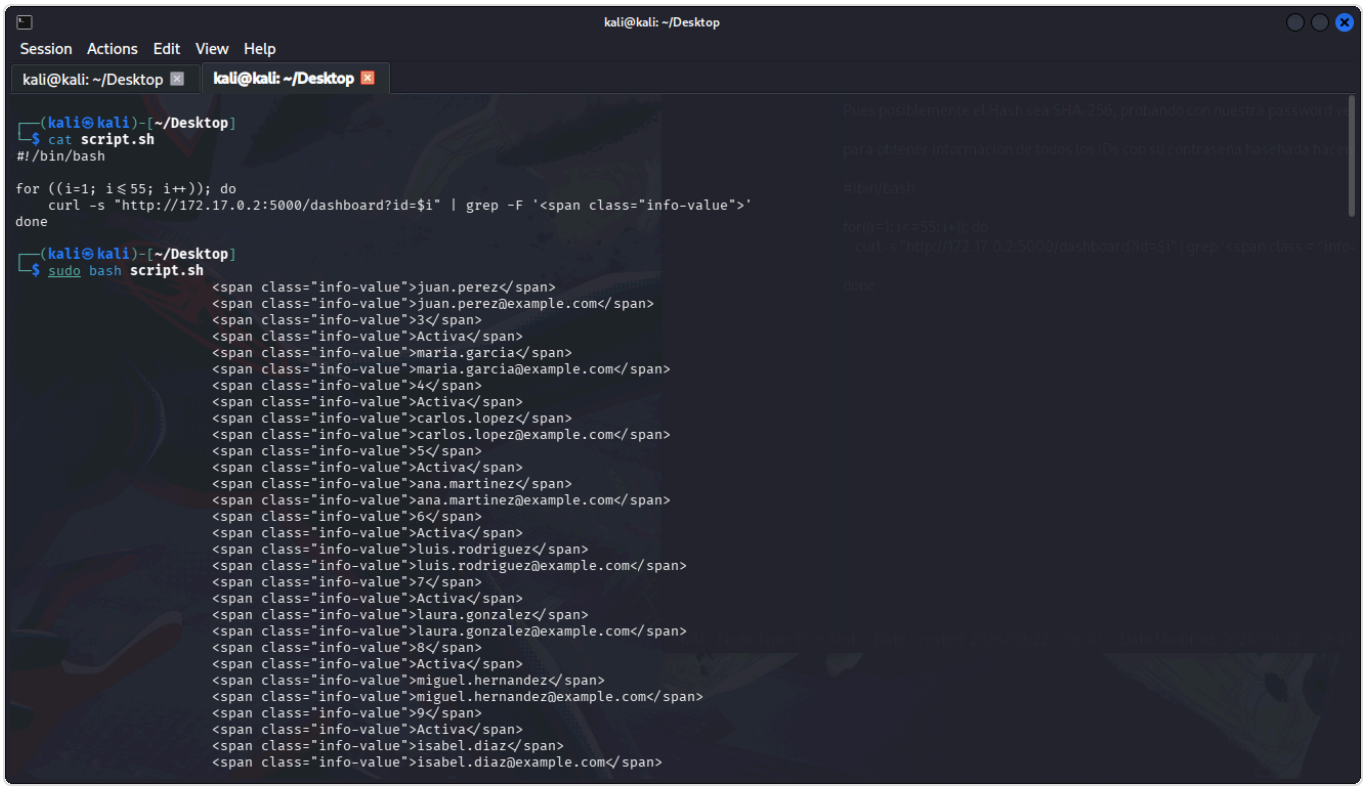
```
echo -n "miclave" | sha256sum
```



Confirmado el formato, tocaba explotar el IDOR a escala. Un bucle simple sobre 55 IDs, extrayendo los `<span class="info-value">` de cada respuesta:

```
#!/bin/bash
for ((i=1; i<=55; i++)); do
    curl -s "http://172.17.0.2:5000/dashboard?id=$i" | grep -F '<span class="info-value">'
done
```

```
sudo bash script.sh
```

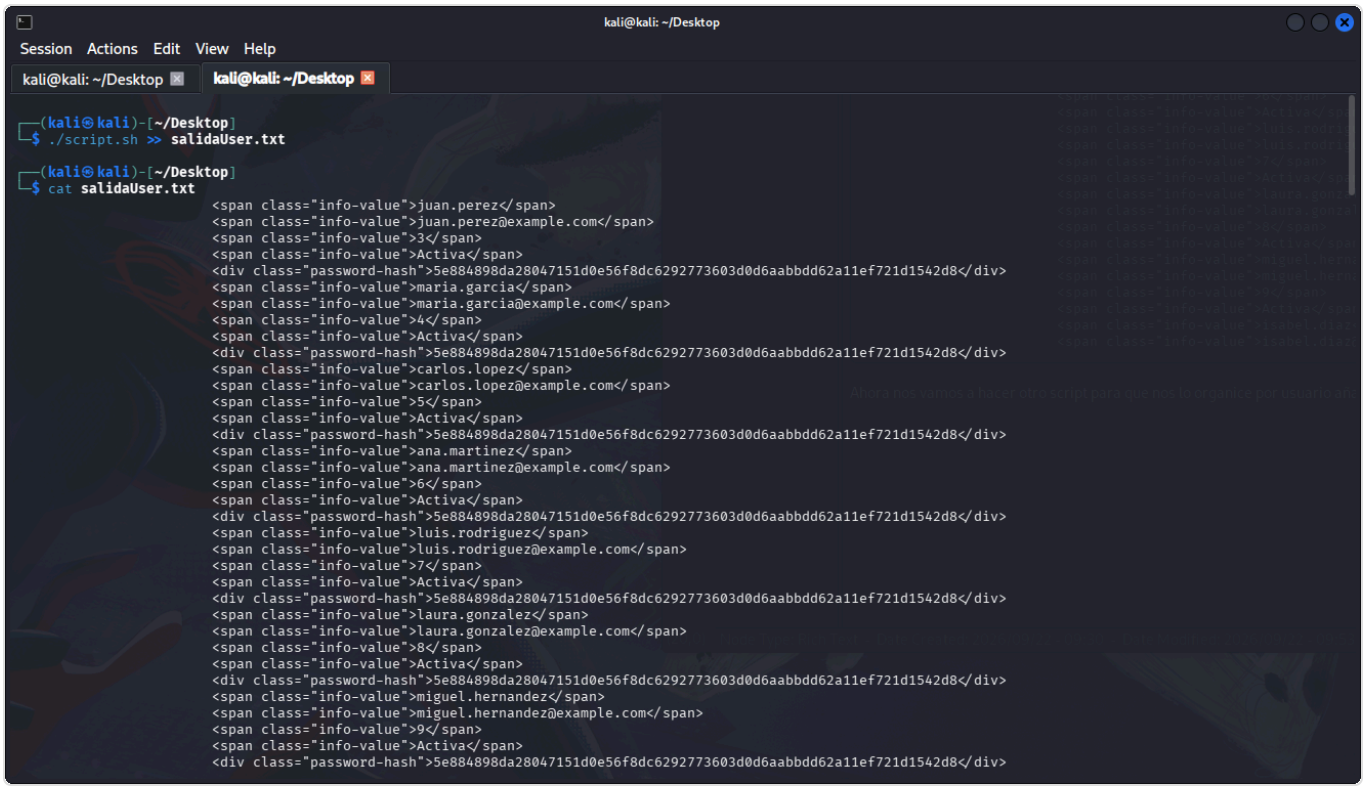


55 peticiones, 55 fichas de usuario filtradas — nombre, email, id, estado, y el hash de la contraseña de cada una — todo sin haber iniciado sesión ni una sola vez.

Limpiando el volcado: de HTML crudo a dos listas utilizables

La salida cruda venía envuelta en etiquetas HTML y mezclaba campos que no interesaban (el email, el estado `Activa`). El objetivo era quedarme solo con **usuario + hash**, en dos ficheros separados para poder atacarlos con John.

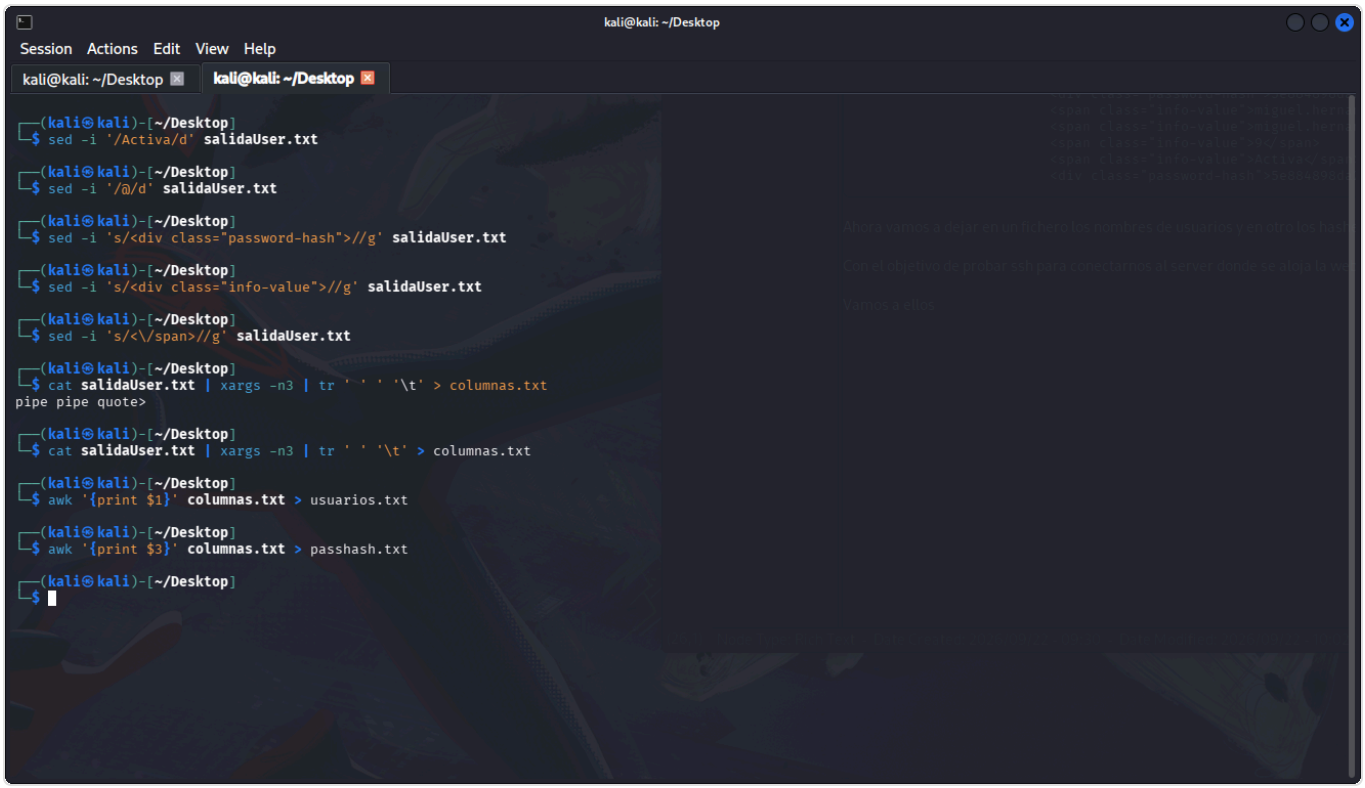
```
./script.sh >> salidaUser.txt
```



Limpieza con `sed` (quitar líneas de estado y de email, y las etiquetas HTML sobrantes) y reorganización en columnas con `xargs` / `tr` / `awk`:

```
sed -i '/Activa/d' salidaUser.txt
sed -i '/@/d' salidaUser.txt
sed -i 's/<div class="password-hash">//g' salidaUser.txt
sed -i 's/<span class="info-value">//g' salidaUser.txt
sed -i 's/<\/span>//g' salidaUser.txt

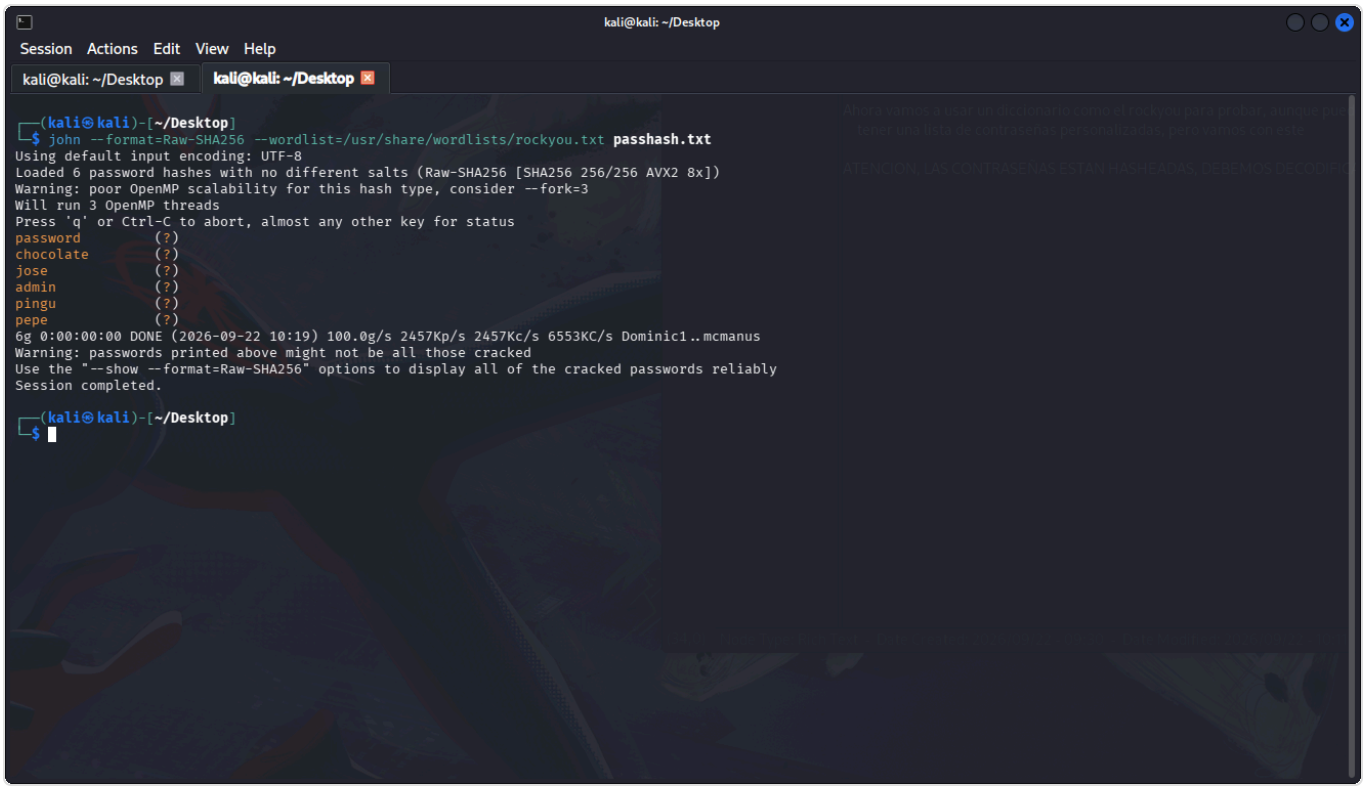
cat salidaUser.txt | xargs -n3 tr ' ' '\t' > columnas.txt
awk '{print $1}' columnas.txt > usuarios.txt
awk '{print $3}' columnas.txt > passhash.txt
```



Resultado: `usuarios.txt` con los nombres de usuario, `passhash.txt` con sus hashes SHA-256 en el mismo orden — listos para cracking en lote.

Cracking masivo con John

```
john --format=Raw-SHA256 --wordlist=/usr/share/wordlists/rockyou.txt passhash.txt
```

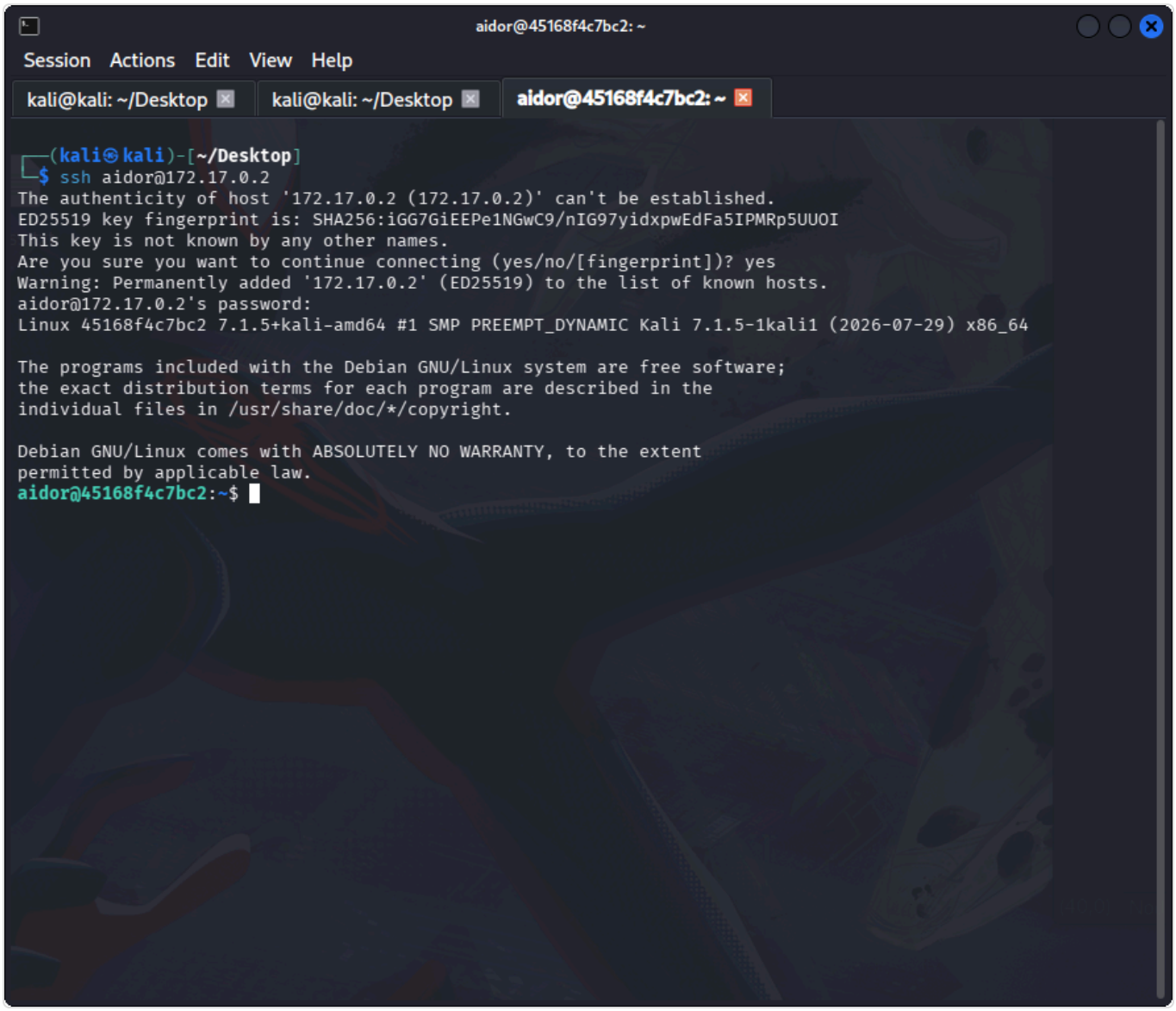


De los hashes filtrados, varios cayeron contra rockyou de forma casi instantánea (`password` , `chocolate` , `jose` , `admin` , `pingu` , `pepe` ...): la lección de siempre — una política de contraseñas débil en una app convierte cualquier fuga de hashes, por pequeña que sea, en credenciales en texto claro en segundos.

Acceso por SSH

Con el listado de usuarios y sus contraseñas ya en claro, tocaba probarlas contra el SSH que ya habíamos visto abierto en el nmap inicial. La combinación `aidor:chocolate` fue válida:

```
ssh aidor@172.17.0.2
```



Callejón sin salida: sudo y SUID

Lo primero al entrar, como siempre, fue mirar los caminos obvios de escalada:

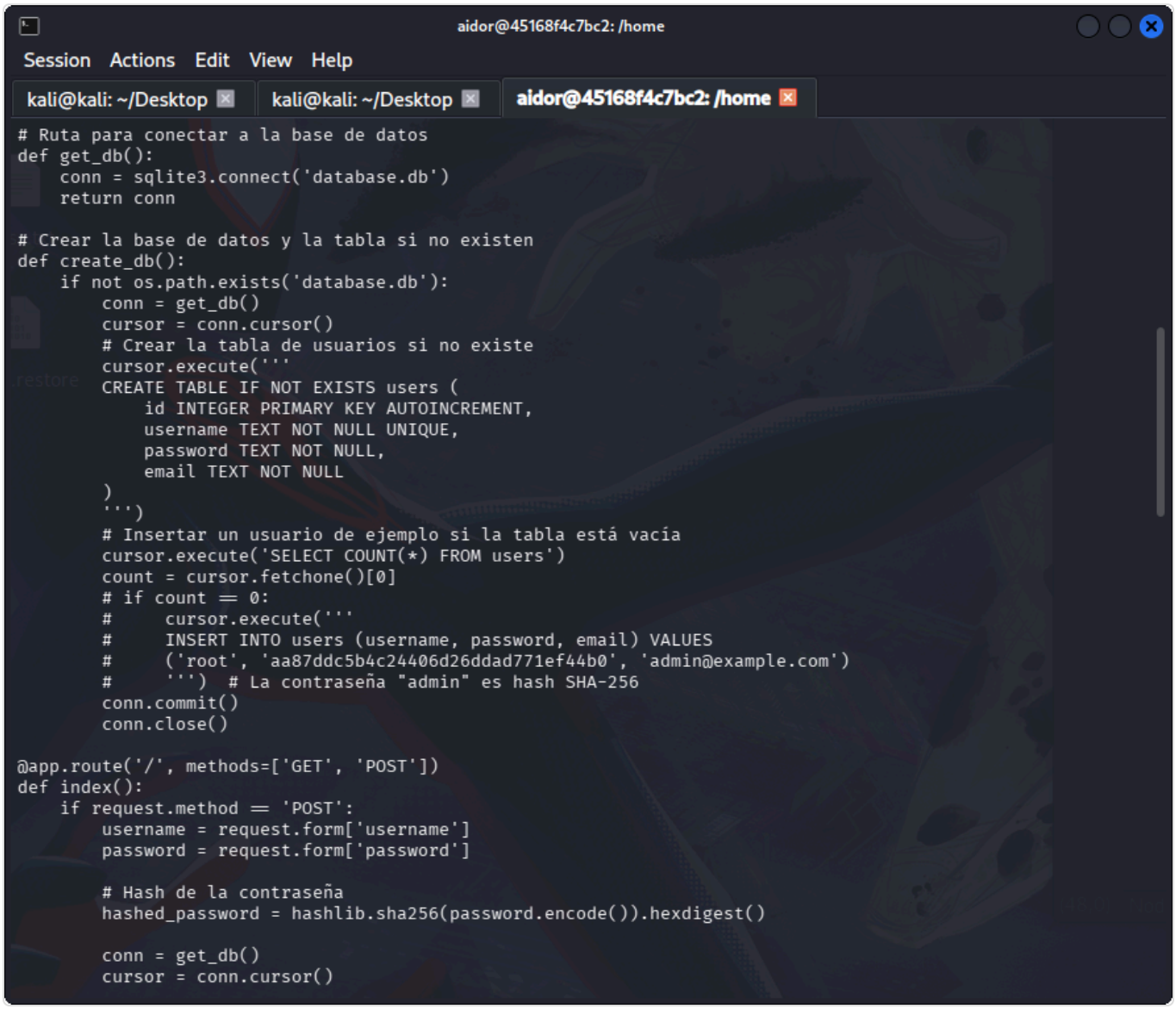
```
sudo -l
find / -perm -4000 -type f 2>/dev/null
```

Ninguno de los dos dio nada aprovechable. Nada de `sudo` sin contraseña, ningún SUID fuera de lo normal. Tocaba mirar qué había en el sistema de archivos en vez de en los permisos.

El código fuente que se dejaron en /home

En `/home` estaban `app.py` y `database.db` — el propio código fuente de la aplicación Flask contra la que habíamos estado atacando por HTTP. Leerlo directamente en el servidor era mucho más rico que cualquier fuzzing:

```
cat app.py
```

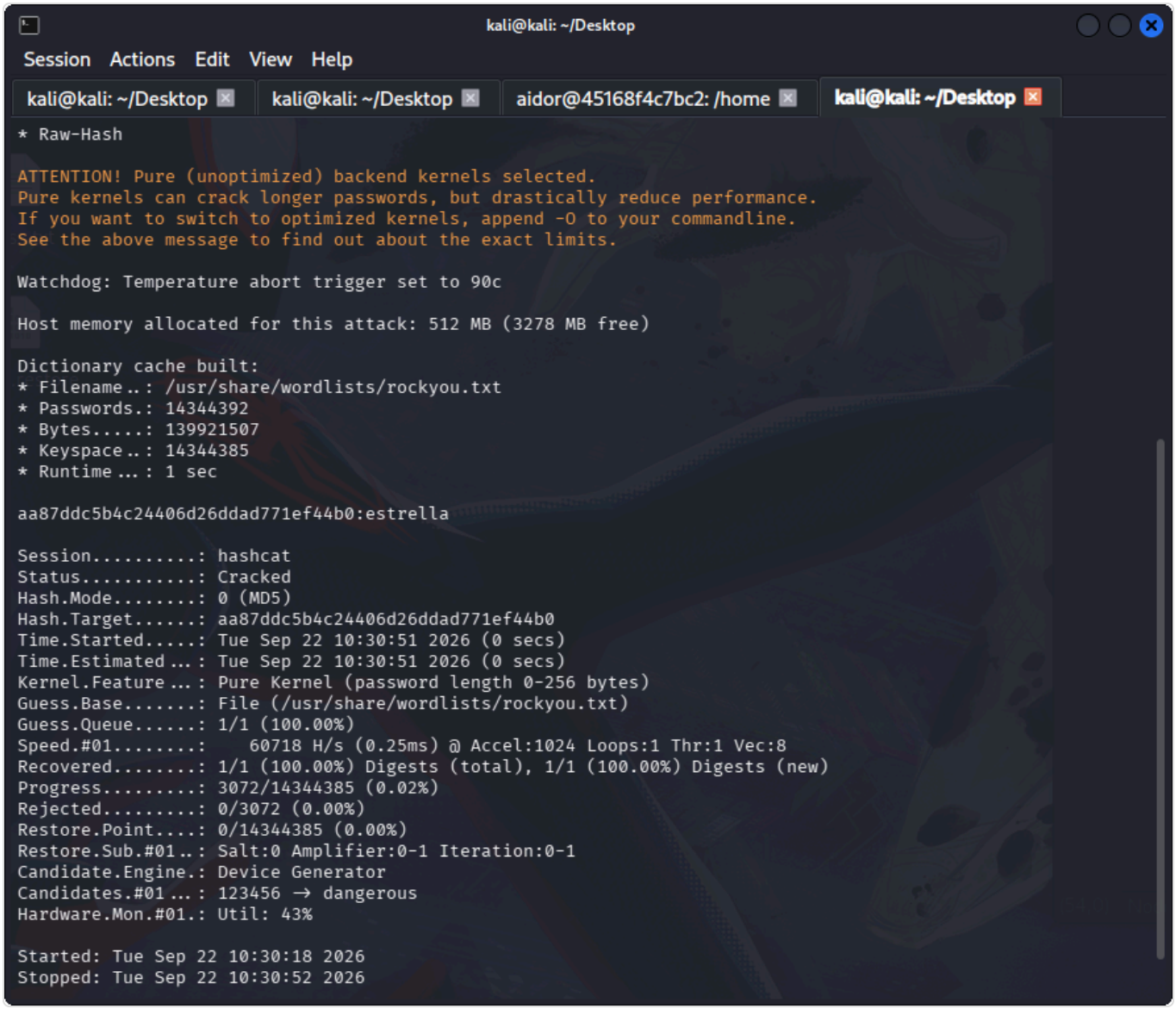


El archivo confirmaba lo que ya sabíamos por fuera (SQLite, tabla `users`, contraseñas hasheadas con `hashlib.sha256()` en el login) y añadía algo que no era visible desde el dashboard: una sentencia `INSERT` comentada, dejada ahí de una versión anterior o de pruebas, con un hash de ejemplo para un usuario `root`. El comentario del propio código lo etiquetaba como "hash SHA-256 de la contraseña admin" — pero el valor tenía solo 32 caracteres hexadecimales, la longitud exacta de un **MD5**, no de un SHA-256 (64 caracteres). El comentario mentía sobre su propio algoritmo.

El hash que no era lo que decía ser

Con la longitud como pista, lo lancé directamente contra Hashcat en modo MD5:

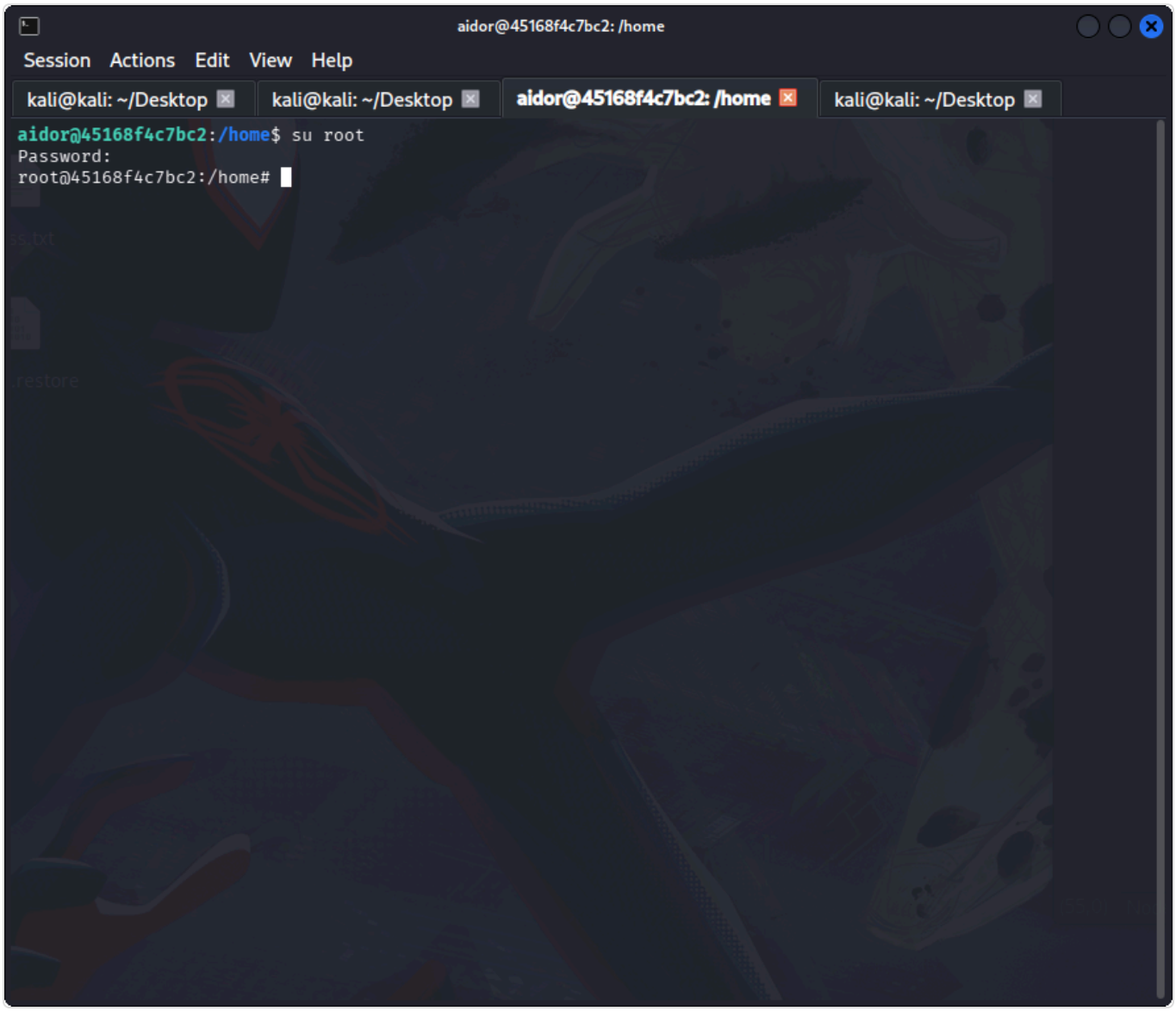
```
hashcat -m 0 aa87ddc5b4c24406d26ddad771ef44b0 /usr/share/wordlists/rockyou.txt
```



Cracked en el mismo segundo: `estrella`.

Root

```
su root
```



La contraseña `estrella` era la de `root` en el sistema, no la de una cuenta `admin` de la web como sugería el comentario del código — la etiqueta del propio desarrollador estaba mal, y confiar en ella a ciegas me habría hecho descartar el hash. Root obtenido.

Lo que me llevo de esta máquina

- **Un IDOR no necesita ser sofisticado para ser crítico.** Un `id` secuencial sin verificación de sesión bastó para volcar la base de datos de usuarios completa, hashes incluidos, sin una sola credencial.
- **Cualquier filtrado de hashes es tan peligroso como la wordlist que tengas enfrente.** Media docena de contraseñas cayeron contra rockyou en segundos — el problema nunca fue la técnica, fue la política de contraseñas.

- **El código fuente de una aplicación es, en la práctica, la documentación más honesta que existe** — más que cualquier comentario dejado en él. Aquí el propio comentario decía "SHA-256" sobre un hash que era MD5; confiar en la longitud real del dato, no en la etiqueta que alguien le puso, fue lo que llevó a la contraseña correcta.
- **Un callejón sin salida (`sudo -l`, SUID) no es el final** — es la señal de que toca mirar en otro sitio: ficheros de configuración, código fuente accesible, backups. La escalada casi nunca es un solo camino.